

Participações — Padrões de Projeto

G7-MonitoreSeuTreino

Requisitos de Arquitetura — UnB/FGA — 22/05/2026

Breve relato sobre as participações e contribuições de cada membro à entrega de padrões de projeto GoF. Todos participaram com comprobatórios.

GoFs Criacionais

Singleton — Módulo de Onboarding

Contexto: Implementação de `OnboardingClassificationRules` como `Singleton` de domínio, centralizando o algoritmo de pontuação de perfil de treino. A instância única garante fonte de verdade única para todos os classificadores.

Membro	Contribuição	Significância	Comprobatórios
Lucas Antunes	Modelagem do Singleton, implementação de <code>calculateScore()</code> e sub-métodos de pontuação por critério (experiência, frequência, técnica, consistência), testes unitários (5 casos), documentação.	Excelente	backend/src/domain/onboarding/rules/

Factory Method — Módulo de Autenticação

Contexto: Encapsulamento dos construtores de `User` e `RefreshToken` em métodos estáticos semânticos. O `create()` é usado para criações genuínas, gerando UUIDs e emitindo eventos de domínio como `UserRegisteredEvent`, enquanto o `reconstitute()` é usado pela persistência para restaurar os objetos sem disparar eventos.

Membro	Contribuição	Significância	Comprobatórios
Samuel Nogueira Caetano	Modelagem do padrão Factory Method nas entidades, blindagem de construtores, implementação de regras e barramentos via Value Objects (<code>UserId</code> , <code>Email</code>), documentação e suíte de testes unitários.	Excelente	backend/src/domain/entities/user.entity.ts, refresh-token.entity.ts e .spec.ts

Factory Method — Módulo de Monitoramento Semanal

Contexto: Criação dinâmica de estratégias de resolução de períodos semanais (`WeeklyPeriodStrategy`) sem acoplar o caso de uso às implementações concretas responsáveis pelo cálculo de datas.

Membro	Contribuição	Significância	Comprobatórios
João Maurício Pilla Nascimento	Modelagem do Factory Method <code>WeeklyPeriodFactory</code> , criação das estratégias <code>CurrentWeekPeriodStrategy</code> e <code>CustomPeriodStrategy</code> , integração com <code>GetWeeklySummaryUseCase</code> e testes manuais com PostgreSQL.	Excelente	backend/src/domain/tracking/services/factory/, strategy/, application/use-cases/tracking/

Builder — Módulo de Exercícios

Contexto: Implementação de `ExerciseBuilder` para garantir a criação correta do agregado `Exercise` com seus `Value Objects` vinculados.

Membro	Contribuição	Significância	Comprobatórios
Daniel Teles	Modelagem do Builder para gerenciar construção de exercícios, separando a lógica inline do Use Case e melhorando a segurança da criação.	Excelente	backend/src/domain/exercises/builders/

Builder — Módulo de Sessão de Treino

Contexto: Implementação de `TrainingSessionBuilder` para permitir a construção incremental e consistente do agregado complexo `TrainingSession`, isolando os use cases das subestruturas internas do `Composite`.

Membro	Contribuição	Significância	Comprobatórios
Eduardo Waski	Modelagem e implementação do <code>TrainingSessionBuilder</code> , API fluida, centralização de validação de invariantes e desenvolvimento de testes unitários.	Excelente	backend/src/domain/builders/training-session.builder.ts e training-session.spec.ts

Builder — Módulo de Usuário

Contexto: Construção validada de comandos para os fluxos RF04 (redefinição de senha) e RF07 (exclusão de conta), garantindo que todos os campos obrigatórios estejam presentes antes da execução da cadeia de responsabilidade.

Membro	Contribuição	Significância	Comprobatórios
André Ricardo Meyer de Melo	Implementação de <code>PasswordResetRequestBuilder</code> e <code>AccountDeletionRequestBuilder</code> , validação de campos obrigatórios, integração com os respectivos use cases.	Excelente	backend/src/presentation/(password-reset-request.builder.ts, account-deletion-request.builder.ts)

Multiton — Módulo de Histórico de Sessões

Contexto: Implementação de `HistoryManager` como `Multiton`, mantendo um pool de gerenciadores por usuário para evitar recriação desnecessária de sessões concluídas a cada requisição.

Membro	Contribuição	Significância	Comprobatórios
Giovanni Dornelas Ferreira	Implementação do <code>Multiton HistoryManager</code> , integração com <code>HistoryService</code> e <code>HistoryObserver</code> , repositório <code>findCompletedByUserId</code> , suporte aos requisitos RF26/RF27 e documentação.	Excelente	backend/src/domain/history/history-manager.ts

Prototype — Módulo de Rotinas

Contexto: Implementação do método `.clone()` dentro do Agregado raiz `Routine` para duplicar fichas com cópia profunda de divisões, emitindo novos IDs e Eventos de Domínio sem delegar manipulação estrutural para o Use Case.

Membro	Contribuição	Significância	Comprobatórios
José Victor Gabriel Menezes da Costa	Modelagem do Prototype na entidade <code>Routine</code> , lógica de cópia profunda de divisões/exercícios e consumo limpo no <code>CloneRoutineUseCase</code> .	Excelente	backend/src/domain/entities/routine.entity.ts e application/use-cases/routines/clone-routine.use-case.ts

GoFs Estruturais

Bridge + Facade — Módulo de Onboarding

Contexto: Bridge: Separação da hierarquia de fluxos (OnboardingFlow) da hierarquia de classificadores (ProfileClassifier). Facade: OnboardingFacade como único ponto de entrada da camada de apresentação para o subsistema de onboarding.

Membro	Contribuição	Significância	Comprobatórios
Lucas Antunes	Bridge: abstração OnboardingFlow, StrengthOnboardingFlow, interface ProfileClassifier, Male/FemaleProfileClassifier, testes (6 casos). Facade: OnboardingFacade, integração com OnboardingController.	Excelente	backend/src/domain/onboarding/bridge/ e presentation/facades/

Decorator + Facade — Módulo de Autenticação

Contexto: Decorator: Empilhamento de CachingUserRepository e LoggingUserRepository sobre o UserPostgresRepository. Facade: AuthenticationFacade como roteador simplificado para o AuthController.

Membro	Contribuição	Significância	Comprobatórios
Samuel Nogueira Caetano	Decorator: wrappers CachingUserRepository e LoggingUserRepository, configuração da montagem de injeção em AuthModule e testes. Facade: refatoração do AuthController e roteamento de sessões.	Excelente	backend/src/infrastructure/database/, modules/auth.module.ts, presentation/facades/ e controllers/auth.controller.ts

Decorator — Módulo de Exercícios

Contexto: Padrões Caching e Logging ao redor do repositório base para ganho em observabilidade e performance sem alterar a interface do repositório concreto.

Membro	Contribuição	Significância	Comprobatórios
Daniel Teles	Adição de decorators CachingExerciseRepository e LoggingExerciseRepository em volta do Postgres Repository, mantendo OCP e SRP.	Excelente	backend/src/infrastructure/database/ e modules/

Proxy — Módulo de Histórico de Sessões

Contexto: Implementação de HistoryServiceProxy para controlar acesso, validar período e registrar logs antes de delegar ao serviço real de histórico.

Membro	Contribuição	Significância	Comprobatórios
Giovanni Dornelas Ferreira	Implementação do HistoryServiceProxy, interface IHistoryService, serviço real HistoryService, wiring em HistoryModule e HistoryController.	Excelente	backend/src/infrastructure/services/history-service.proxy.ts e application/services/history.service.ts

Facade — Módulo de Usuário

Contexto: Interface única para orquestrar a cadeia de responsabilidade, repositórios, envio de e-mail e publicação de eventos nos fluxos RF04 e RF07.

Membro	Contribuição	Significância	Comprobatórios
André Ricardo Meyer de Melo	Implementação de PasswordResetFacade e AccountDeletionFacade, orquestração da cadeia, repositórios, e-mail e eventos de domínio.	Excelente	backend/src/presentation/facades/ (password-reset.facade.ts, account-deletion.facade.ts)

Composite — Módulo de Sessão de Treino

Contexto: Organização da hierarquia de treinos tratando séries (*TrainingSet* — *Leaf*) e exercícios (*ExerciseNode* — *Composite*) de forma uniforme sob a interface *WorkoutComponent*.

Membro	Contribuição	Significância	Comprobatórios
Eduardo Waski	Modelagem e implementação do Composite: interface <i>WorkoutComponent</i> , classe <i>ExerciseNode</i> e classe <i>TrainingSet</i> , e testes de cálculo recursivo.	Excelente	<code>backend/src/domain/entities/workout-component.ts</code> , <code>exercise-node.ts</code> , <code>training-set.ts</code>

Proxy — Módulo de Rotinas

Contexto: Intercepção das chamadas de persistência da ficha de treino através de um *RoutineRepositoryProxy* que detém o mesmo contrato do repositório real.

Membro	Contribuição	Significância	Comprobatórios
José Victor Gabriel Menezes da Costa	Criação do Proxy protetor, mantendo o ORM do TypeORM livre de implementações lógicas na camada de infraestrutura.	Excelente	<code>backend/src/infrastructure/proxies/routine-repository.proxy.ts</code>

Facade — Módulo de Monitoramento Semanal

Contexto: Simplificação do acesso ao subsistema de monitoramento semanal, encapsulando a comunicação entre controller, caso de uso e regras de domínio em interface única.

Membro	Contribuição	Significância	Comprobatórios
João Maurício Pilla Nascimento	Implementação do <i>TrackingFacade</i> , isolamento do <i>TrackingController</i> , padronização das respostas via <i>TrackingViewModel</i> e integração com autenticação JWT.	Excelente	<code>backend/src/presentation/facades/tracking.facade.ts</code> , <code>controllers/tracking.controller.ts</code> , <code>view-models/tracking.view-model.ts</code>

GoFs Comportamentais

Memento + Template Method — Módulo de Onboarding

Contexto: Memento: Captura do estado do perfil antes de um redo, preservando histórico completo. Template Method: Esqueleto imutável do fluxo de classificação em `OnboardingFlow.execute()`, com hooks `beforeClassify()` e `afterClassify()`.

Membro	Contribuição	Significância	Comprobatórios
Lucas Antunes	Memento: <code>OnboardingMementoVO</code> , <code>createMemento()</code> em <code>TrainingProfile</code> , <code>RedoOnboardingUseCase</code> (caretaker), <code>onboarding_history</code> (ORM + repositório), testes (5 casos). Template Method: <code>execute()</code> em <code>OnboardingFlow</code> , hooks protegidos, <code>StrengthOnboardingFlow</code> .	Excelente	<code>backend/src/domain/onboarding/entities/</code> , <code>value-objects/</code> , <code>bridge/</code> , <code>infrastructure/database/</code>

Template Method + Observer — Módulo de Autenticação

Contexto: Template Method: Esqueleto de execução na classe base `UseCase` com três fases invariantes. Observer: `DomainEventBus` para assinar e despachar eventos como `UserRegisteredEvent`.

Membro	Contribuição	Significância	Comprobatórios
Samuel Nogueira Caetano	Template Method: modelagem do <code>UseCase</code> genérico abstrato, hook <code>registerAggregate</code> , rotina recursiva <code>collectAggregates</code> . Observer: infraestrutura de acúmulo no <code>AggregateRoot</code> , criação do <code>DomainEventBus</code> com <code>allSettled</code> e testes.	Excelente	<code>backend/src/application/use-cases/bas</code> <code>e.use-case.ts</code> , <code>events/domain-event-bus.ts</code> , <code>domain/entities/aggregate-root.ts</code>

Observer — Módulo de Histórico de Sessões

Contexto: Notificação automática após registro de sessão de treino. O `WorkoutSessionSubject` notifica o `HistoryObserver` para atualizar histórico sem acoplamento.

Membro	Contribuição	Significância	Comprobatórios
Giovanni Dornelas Ferreira	Observer: <code>WorkoutSessionSubject</code> , <code>HistoryObserver</code> , <code>SessionObserver</code> , integração em <code>RegisterSessionUseCase.notify()</code> , inscrição em <code>HistoryModule.onModuleInit</code> .	Excelente	<code>backend/src/domain/history/observers/</code> e <code>application/use-cases/session/regist</code> <code>r-session.use-case.ts</code>

Chain of Responsibility — Módulo de Exercícios

Contexto: Construção elástica da query de busca através de pipeline de handlers para encadear filtros dinâmicos.

Membro	Contribuição	Significância	Comprobatórios
Daniel Teles	Encadeamento de restrições de busca em <code>ExerciseSearchChain</code> para permitir múltiplos testes dinâmicos de filtro.	Excelente	<code>backend/src/infrastructure/database/ex</code> <code>ercise-search.chain.ts</code>

Chain of Responsibility — Módulo de Usuário

Contexto: Etapas sequenciais de validação e execução nos fluxos RF04 (4 handlers) e RF07 (4 handlers), com aborto silencioso por segurança.

Membro	Contribuição	Significância	Comprobatórios
André Ricardo Meyer de Melo	Implementação das cadeias RF04 e RF07 com decisões de segurança embutidas, integração com builders e facades.	Excelente	<code>backend/src/application/</code> <code>(password-reset.chain.ts,</code> <code>account-deletion.chain.ts)</code>

Iterator — Módulo de Sessão de Treino

Contexto: Implementação do `TrainingSetIterator` para navegar pela árvore recursiva do `Composite` e expor iteração sequencial e linear das séries.

Membro	Contribuição	Significância	Comprobatórios
Eduardo Waski	Implementação da interface <code>Iterator</code> , do concreto <code>TrainingSetIterator</code> (achatamento recursivo na construção) e testes unitários de travessia.	Excelente	<code>backend/src/domain/iterators/</code> e <code>domain/entities/training-session.ts</code>

Mediator — Módulo de Rotinas

Contexto: O `ActivateRoutineUseCase` comunica-se com o `DomainEventBus` (mediador), que invoca o `DeactivateOtherRoutinesHandler` em paralelo para garantir exclusividade de ficha ativa.

Membro	Contribuição	Significância	Comprobatórios
José Victor Gabriel Menezes da Costa	Implementação do <code>Handler</code> reativo às ativações, depuração crítica no <code>Event Bus</code> e conversão segura de entidades contra repositórios anêmicos.	Excelente	<code>backend/src/application/events/handler</code> <code>s/deactivate-other-routines.handler.ts</code> e <code>events/domain-event-bus.ts</code>

Strategy — Módulo de Monitoramento Semanal

Contexto: Cálculo de métricas derivadas das sessões concluídas (total de sessões e dias distintos) sem concentrar múltiplos algoritmos no caso de uso.

Membro	Contribuição	Significância	Comprobatórios
João Maurício Pilla Nascimento	Modelagem das estratégias <code>TotalSessionsStrategy</code> e <code>ActiveDaysStrategy</code> , implementação das concretas <code>Default*</code> , composição via <code>DefaultWeeklySummaryCalculator</code> e integração com <code>NestJS</code> .	Excelente	<code>backend/src/domain/tracking/services/weekly-summary.service.ts</code> , <code>strategy/weekly-summary.strategy.ts</code> , <code>application/use-cases/tracking/</code>

Histórico de Versões

Versão	Data	Descrição	Autor
1.0	19/05/2026	Registro de participações nos padrões GoF do módulo de Onboarding.	Lucas Antunes
1.1	20/05/2026	Registro de participações nos padrões GoF do módulo de Autenticação.	Samuel Nogueira Caetano
1.2	20/05/2026	Registro de participações nos padrões GoF do módulo de Histórico de Sessões.	Giovanni Dornelas Ferreira
1.3	21/05/2026	Registro de participações nos padrões GoF do módulo de Exercícios.	Daniel Teles
1.4	21/05/2026	Registro de participações nos padrões GoF do módulo de Usuário.	André Ricardo Meyer de Melo
1.5	21/05/2026	Registro de participações nos padrões GoF do módulo de Sessão de Treino.	Eduardo Waski
1.6	21/05/2026	Registro de participações nos padrões GoF do módulo de Rotinas.	José Victor Gabriel Menezes da Costa
1.7	21/05/2026	Registro de participações nos padrões GoF do módulo de Monitoramento Semanal.	João Maurício Pilla Nascimento
1.8	22/05/2026	Consolidação: unificação, padronização e reorganização por categoria GoF.	Lucas Antunes